

Web technológiák 2. jegyzőkönyv

Smart Pantry

Nyakó Sándor

PHRO7R

Miskolci Egyetem

Miskolc, 2026. május 11.

Tartalomjegyzék

1. Bevezetés.....	3
2. Projekt áttekintés.....	3
2.1 Funkcionális követelmények.....	3
2.1.1 Alapanyag-kezelés (CRUD).....	3
2.1.2 Lejárat-követés és analitika.....	3
2.1.3 Bevásárlólista és receptjavaslatok.....	3
2.2 Nem funkcionális követelmények.....	4
3. Technológiai Stack.....	4
3.1 Core technológiák.....	4
3.1.1 Node.js és TypeScript (Backend).....	4
3.1.2 Angular 21 (Frontend).....	4
3.1.3 MEAN stack architektúra (MVC/MVVM).....	5
3.2 Adatkezelés.....	5
3.2.1 MongoDB és Mongoose ODM.....	5
3.2.2 JWT alapú hitelesítés.....	5
3.3 UI és stílusok.....	5
3.3.1 Tailwind CSS v4.....	5
3.3.2 Angular Signals és ReactiveForms.....	6
4. Projekt struktúra.....	6
4.1 server.ts és auth.controller.ts — belépési pont és hitelesítés.....	6
4.2 AuthService.ts — kliensoldali hitelesítési állapotkezelés.....	8
4.3 DashboardComponent.ts — fő nézet és állapotkezelés.....	9
4.4 Ingredient.ts és User.ts — Mongoose modellek.....	10
5. Képernyők.....	11
5.1 Bejelentkezés és regisztráció.....	11
5.2 Főoldal (Dashboard).....	12
5.3 Alapanyag hozzáadása — modális ablak.....	12
5.4 Bevásárlólista (Shopping List).....	13
5.5 Receptjavaslatok.....	14
5.6 Toast értesítések.....	14

1. Bevezetés

A Smart Pantry egy teljes körű, webalapú alkalmazás, amelynek célja az otthoni élelmiszerkészlet digitális nyilvántartásának egyszerűsítése és automatizálása. Az alkalmazás lehetővé teszi a felhasználók számára alapanyagaik valós idejű követését, lejárat dátumok figyelemmel kísérését, automatikus bevásárlólista generálást, valamint mesterséges intelligencia alapú receptjavaslatokat a rendelkezésre álló hozzávalók alapján.

Az adatok perzisztens tárolását MongoDB NoSQL adatbázis végzi. Az alkalmazás a MEAN stack architektúrát (MongoDB, Express.js, Angular, Node.js) követi, amelyet TypeScript típusbiztos kódolással egészítenek ki. A megjelenítés modern, mobilbarát kialakítást követ Tailwind CSS v4 segítségével, az állapotkezelés Angular Signals és reaktív RxJS adatfolyamok kombinációján alapul.

2. Projekt áttekintés

2.1 Funkcionális követelmények

2.1.1 Alapanyag-kezelés (CRUD)

- **Elem hozzáadása:** Név, kategória, mennyiség, egység, kalória, lejárat dátum és minimális küszöbérték megadásával, reaktív modális ablakban
- **Szerkesztés:** Meglévő alapanyagok módosítása PUT /api/ingredients/:id végponton keresztül
- **Törlés:** Alapanyagok eltávolítása megerősítő dialógussal, azonnali optimista UI frissítéssel
- **Vásárolt státusz / feltöltés:** A bevásárlólistán 'Mark as Purchased' gombbal a mennyiség a minimális küszöbre állítható, és az elem eltűnik a listáról
- **Kategóriák:** Szabad szöveges kategória-megadás, doughnut diagramon vizualizálva

2.1.2 Lejárat-követés és analitika

- **Lejárat-figyelés:** Háromszintű vizuális kódolás: normál (zöld pont) / figyelmeztetés (sárga, 7 napon belül) / lejárt (piros)
- **Statisztika-kártyák:** Az összes alapanyag száma és a hamarosan lejáró tételek száma a dashboard tetején
- **Kategória-diagram:** Doughnut chart az alapanyagok kategória szerinti eloszlásáról (ng2-charts + Chart.js)

2.1.3 Bevásárlólista és receptjavaslatok

- **Automatikus bevásárlólista:** GET /api/shopping-list — azokat az alapanyagokat listázza, amelyek mennyisége a beállított minimális küszöb alá esett
- **Receptjavaslatok:** GET /api/recipes/suggest — Spoonacular API integráció: a kamra tartalmából recepteket javasol, megjeleníti a hiányzó hozzávalók számát
- **Receptek megnyitása:** A receptkártyákra kattintva a Spoonacular részletes receptoldala nyílik meg

2.2 Nem funkcionális követelmények

A rendszer nem-funkcionális követelményei négy fő területre oszthatók: teljesítmény, használhatóság, biztonság és karbantarthatóság. A teljesítmény szempontjából az Angular ChangeDetectionStrategy.OnPush és a Signals-alapú állapotkezelés minimalizálja a felesleges változásdetektálási ciklusokat, az adatbázis-lekérdezések Mongoose-szintű indexeléssel optimalizáltak.

A használhatóságot reszponzív Tailwind CSS elrendezés biztosítja, amely mobilon, tableten és asztali böngészőkön egyaránt optimálisan jelenik meg, animált toast-értesítésekkel és modális ablakokkal. A biztonsági követelmények teljesítéséhez az alkalmazás bcrypt jelszó-titkosítást (10 sókör), JWT-alapú állapotmentes hitelesítést, és Mongoose paraméteres lekérdezéseket alkalmaz a NoSQL injection ellen. A karbantarthatóság érdekében az alkalmazás Clean Code, DRY és SOLID elveket követ, szigorú TypeScript típusozással és moduláris, MVVM-szerű komponens-architektúrával.

3. Technológiai Stack

3.1 Core technológiák

3.1.1 Node.js és TypeScript (Backend)

A backend Node.js v20 LTS futtatókörnyezeten alapul, Express.js v4 HTTP keretrendszerrel. Az összes backendkód TypeScript 5.6-ban íródott, amelyet tsx watch fordít futtatás közben fejlesztői módban. A TypeScript több előnyt nyújt: null-safety, szigorú interfész-alapú típusozás, kompilációs idejű hibakeresés. A belépési pont (server.ts) regisztrálja az összes Express route-ot, globális hibakezelő middleware-t és az adatbázis-kapcsolatot.

Fontosabb Node.js / Express funkciók:

- **asyncHandler wrapper:** try/catch mentes async route kezelők
- **errorHandler middleware:** egységesített hibaválasz-formátum (message + statusCode)
- **CORS middleware:** Angular devszerver (localhost:4200) engedélyezése
- **requireAuth middleware:** JWT-ellenőrzés védett végpontokon
- **express.json():** JSON request body automatikus parseolása

3.1.2 Angular 21 (Frontend)

A frontend Angular 21-ben készült, kizárólag önálló (standalone) komponensekkel, Angular CLI által generált projekt-struktúrában. Az Angular router lusta betöltést (lazy loading) alkalmaz minden feature-modulnál, csökkentve a kezdeti bundle-méretet. A függőség-injekció szigorúan az inject() függvényen keresztül történik, a konstruktor-injekció mellőzésével.

Fontosabb Angular funkciók:

- **Angular Signals és computed():** reaktív állapotkezelés, OnPush optimalizálással
- **takeUntilDestroyed():** automatikus RxJS feliratkozás-leállítás komponens megszűnésekor
- **CanActivateFn (authGuard):** funkcionális route-védelmi guard
- **HttpInterceptorFn (authInterceptor):** automatikus Bearer token csatolása HTTP kérésekhez
- **ReactiveForms + strictNumber validator:** típusbiztos form-validáció egyedi validátorral

3.1.3 MEAN stack architektúra (MVC/MVVM)

A MEAN stack architektúra lehetővé teszi a kód rétegenkénti szétválasztását és tesztelhetőségét. A backend MVC mintát követ: a Controller réteg kezeli a HTTP kérés-válasz ciklust, a Model réteg (Mongoose) az adatbázis-sémákat és validációt, a Service réteg (pl. RecipeService) a külső API hívásokat. A frontend MVVM szerkezetű: a Template (View) csak a megjelenítést végzi, a Component (ViewModel) az állapotot és eseményeket kezeli, az Angular Service (Model) az adatelérést biztosítja.

3.2 Adatkezelés

3.2.1 MongoDB és Mongoose ODM

Az adatok tárolásához MongoDB 7 NoSQL dokumentumorientált adatbázist alkalmazunk, amelyet a Mongoose 8.5 ODM réteg absztrahálja. A Mongoose típusbiztos sémadefiníciókat, automatikus validációt és Mongoose middleware-eket (pre-hook) biztosít minimális boilerplate kóddal. Az adatbázis-kapcsolat a singleton connectDB() függvényen keresztül épül fel, és a szerver indulásánál azonnal ellenőrzésre kerül.

Fontosabb Mongoose funkciók:

- **Schema + timestamps: true:** createdAt/updatedAt automatikus kezelés
- **unique: true index:** névazonosság garantálása adatbázis szinten
- **Number.isFinite validator:** stringből konvertált, végtelen számok kiszűrése
- **select: false (password):** jelszó-hash soha nem kerül vissza a lekérdezésekben
- **pre('save') hook:** jelszó automatikus bcrypt titkosítása mentés előtt

3.2.2 JWT alapú hitelesítés

A hitelesítés JSON Web Token (JWT) alapú, állapotmentes munkamenet-kezelést alkalmaz. A token a bejelentkezés vagy regisztráció után kerül kibocsátásra, és a kliens localStorage-ban tárolja. Minden védett API-kéréshez a token az Authorization: Bearer <token> fejlécbe kerül, amelyet a requireAuth middleware ellenőriz. A JWT titkosítókulcsot (JWT_SECRET) a .env fájl tartalmazza; a getSecret() segédfüggvény az alkalmazás indításakor ellenőrzi annak meglétét (fail-fast elv).

3.3 UI és stílusok

3.3.1 Tailwind CSS v4

A felhasználói felület kialakítása Tailwind CSS v4-gyel történt, amely az Angular PostCSS build-folyamatba (@tailwindcss/postcss) van integrálva. A dizájntokenek (márkacolor, semleges palette, szemantikus színek) a styles.scss @theme blokkjában kerülnek definiálásra CSS custom property-ként, amelyek az egész alkalmazásban konzisztensen újrafelhasználhatók. Az @layer components rétegben újrafelhasználható osztályok (btn-primary, form-input, badge, nav-link) kapnak helyet, csökkentve a template-kódban lévő ismétlődést.

3.3.2 Angular Signals és ReactiveForms

Az alkalmazás Signal-alapú reaktív állapotkezelést alkalmaz a hagyományos BehaviorSubject megközelítés helyett. A privát `_ingredients` `signal()` tárolja a nyers adatokat, míg a `computed()` függvénnyel levezetett állapotok (`isEmpty`, `rowCount`, `expiringSoonCount`, `expiryStatusMap`) automatikusan frissülnek bármely forrásadat-változáskor, anélkül hogy manuális feliratkozás-kezelés szükséges lenne.

A form-validáció Angular ReactiveForms-szal valósul meg. Az egyedi `strictNumber` validator TypeScript-ben ellenőrzi, hogy a `type='text'` beviteli mezőkbe beírt érték valóban véges szám-e, és opcionálisan kizárja a negatív értékeket. A validációs hibák csak az érintett (`touched`) mezőknél jelennek meg, Tailwind piros keret és hibaüzenet formájában.

4. Projekt struktúra

4.1 server.ts és auth.controller.ts — belépési pont és hitelesítés

A `server.ts` az Express alkalmazás belépési pontja, amely inicializálja az összes szükséges komponens: middleware-eket, Mongoose modelleket, route-okat, hibakezelőt és az adatbázis-kapcsolatot. A Mongoose modellek importálása a route-kezelők regisztrálása előtt kötelező, mivel a Mongoose az indexeket az első kapcsolódáskor építi fel.

`server.ts` — route-regisztráció és middleware-lánc

```
// server.ts — Az alkalmazás belépési pontja
const app = express();
app.use(cors());
app.use(express.json());

// Fejléclogger minden beérkező kérésre
app.use((req, _res, next) => {
  console.log(`[${new Date().toISOString()}] ${req.method} ${req.originalUrl}`);
  next();
});

app.use('/api/auth', authRoutes);
app.use('/api/ingredients', ingredientsRoutes);
app.use('/api/shopping-list', shoppingListRoutes);
app.use('/api/recipes', recipesRoutes);
app.use(errorHandler);
```

Az `auth.controller.ts` a regisztrációs és bejelentkezési logikát tartalmazza. A `register()` metódus `User.create()` segítségével új felhasználót hoz létre; a Mongoose `pre('save')` hook automatikusan hasheli a jelszót. A `login()` metódus megkeresi a felhasználót e-mail cím alapján — a jelszó mezőt az explicit `.select('+password')` hívással be kell kérni, mivel az alapértelmezetten ki van zárva a lekérdezésekből. Sikeresen egyező jelszó esetén JWT tokenet bocsát ki.

auth.controller.ts — login() metódus

```
// auth.controller.ts — bejelentkezési logika
export const login = async (req: Request, res: Response): Promise<void> => {
  const { email, password } = req.body as AuthBody;

  if (!email || !password) {
    res.status(400).json({ message: 'email and password are required' });
    return;
  }

  const user = await User.findOne({ email: email.toLowerCase() })
    .select('+password');

  if (!user || !(await user.comparePassword(password))) {
    res.status(401).json({ message: 'Invalid email or password' });
    return;
  }

  res.status(200).json(
    toAuthResponse({ id: user.id, name: user.name, email: user.email }),
  );
};
```

A `requireAuth` middleware minden védett végpont előtt fut. Kinyeri a Bearer token az Authorization fejlécből, `verifyToken()`-nel ellenőrzi, majd a dekódolt payload-ot (`userId`, `email`) csatolja a `req.user` objektumhoz a downstream route-kezelők számára.

auth.middleware.ts — requireAuth()

```
// auth.middleware.ts
export const requireAuth = (req: Request, res: Response, next: NextFunction): void => {
  const header = req.headers.authorization;
  if (!header || !header.startsWith('Bearer ')) {
    res.status(401).json({ message: 'Authentication token is missing' });
    return;
  }
  const token = header.slice('Bearer '.length).trim();
  try {
    const payload = verifyToken(token);
    req.user = { userId: payload.userId, email: payload.email };
    next();
  } catch {
    res.status(401).json({ message: 'Invalid or expired token' });
  }
};
```

4.2 AuthService.ts — kliensoldali hitelesítési állapotkezelés

Az AuthService az Angular alkalmazás globális hitelesítési service-e (providedIn: 'root'). Signal-alapú reaktív állapotot tart fenn a token és a bejelentkezett felhasználó számára. A _token és _user privát signal-ok; az isLoggedIn computed signal automatikusan igazra értékelődik, amint érvényes token van jelen. A munkamenet adatai localStorage-ban perzisztálnak az oldal-újrátöltések között.

auth.service.ts — Signal-alapú hitelesítés

```
// auth.service.ts — Signal-alapú állapotkezelés
@Inject({ providedIn: 'root' })
export class AuthService {
  private readonly _token = signal<string | null>(
    localStorage.getItem(TOKEN_KEY)
  );
  private readonly _user = signal<AuthUser | null>(
    AuthService.parseStoredUser()
  );

  readonly currentUser = this._user.asReadonly();
  readonly isLoggedIn = computed(() => this._token() !== null);

  login(payload: LoginRequest): Observable<AuthResponse> {
    return this.http
      .post<AuthResponse>(`${environment.apiUrl}/auth/login`, payload)
      .pipe(tap((res) => this.persistSession(res)));
  }

  logout(): void {
    localStorage.removeItem(TOKEN_KEY);
    this._token.set(null);
    this._user.set(null);
    void this.router.navigate(['/login']);
  }
}
```

Az authInterceptor funkcionális HTTP interceptor, amely automatikusan csatolja a Bearer tokent minden kimenő HTTP kéréshez. Csak akkor módosítja a kérést, ha érvényes token áll rendelkezésre és nincs már meglévő Authorization fejléc.

4.3 DashboardComponent.ts — fő nézet és állapotkezelés

A DashboardComponent az alkalmazás fő nézete, amely kezeli az alapanyag-lista megjelenítését, törlési műveleteket, lejárat-figyelést, analitikát és receptjavaslatokat. A komponens ChangeDetectionStrategy.OnPush stratégiát alkalmaz az Angular változásdetektálás optimalizálásához.

Főbb signal-ok és computed signal-ok:

- **_ingredients signal()**: a nyers alapanyag-lista privát, írható állapota
- **rowCount computed()**: az alapanyagok száma, isEmpty-t erre alapozza
- **isEmpty computed()**: üres állapot meghatározása (betöltés után, 0 elem)
- **expiringSoonCount computed()**: 7 napon belül lejáró tételek darabszáma
- **expiryStatusMap computed()**: Map<id, ExpiryStatus> — O(1) lejárat-kiolvasás a template-ből

dashboard.component.ts — Computed signal-ok és expiryStatusMap

```
// dashboard.component.ts — Computed signal-ok
readonly rowCount = computed(() => this._ingredients().length);
readonly isEmpty = computed(() => !this.isLoading() && this.rowCount() === 0);
readonly expiringSoonCount = computed(
  () => this._ingredients()
    .filter((i) => this.getExpiryStatus(i._id) === 'expiring').length,
);

// Pre-számított lejárat-térkép, hogy a template ne számoljon minden CD-ciklusban
readonly expiryStatusMap = computed<ReadonlyMap<string, ExpiryStatus>>>(() => {
  const map = new Map<string, ExpiryStatus>();
  for (const ingredient of this._ingredients()) {
    map.set(ingredient._id, this.computeExpiryStatus(ingredient.expirationDate));
  }
  return map;
});
```

Főbb metódusok:

`loadIngredients()` — `IngredientService.getAll()` Observable-t iratkoz fel, `takeUntilDestroyed(destroyRef)`-fel védi a memóriaszivárgástól. Sikeres betöltésnél `_ingredients.set(list)`, hiba esetén `errorMessage` signal-t állít.

dashboard.component.ts — deleteIngredient() optimista lista-frissítéssel

```
// dashboard.component.ts — Törlési logika
deleteIngredient(ingredient: Ingredient): void {
  if (!confirm('Remove "${ingredient.name}" from your pantry?')) return;
  this.deletingId.set(ingredient._id);
  this.ingredientService.delete(ingredient._id)
    .pipe(takeUntilDestroyed(this.destroyRef))
    .subscribe({
      next: () => {
        this._ingredients.set(
          this._ingredients().filter((i) => i._id !== ingredient._id)
        );
        this.deletingId.set(null);
        this.toast.show(`${ingredient.name} eltávolítva.`, 'success');
      },
      error: () => { this.toast.show('Törlés sikertelen.', 'error'); },
    });
}
```

4.4 Ingredient.ts és User.ts — Mongoose modellek

Az `Ingredient.ts` Mongoose sémája nyolc mezőt definiál. A `quantity` és `calories` mezőknél egyedi `Number.isFinite()` validator ellenőrzi, hogy a Mongoose string-ből számra konvertált érték véges-e — ez kiszűri az 'abc' vagy Infinity eseteket. A `timestamps: true` opció automatikusan kezeli a `createdAt` és `updatedAt` mezőket.

models/Ingredient.ts — Mongoose séma (Number.isFinite validátorokkal)

```
// models/Ingredient.ts — Mongoose séma
const ingredientSchema = new Schema<Ingredient>({
  name: { type: String, required: true, unique: true,
    minlength: 2, maxlength: 80, trim: true },
  category: { type: String, required: true, maxlength: 50 },
  unit: { type: String, required: true, default: 'pcs' },
  quantity: { type: Number, required: true, min: 0,
    validate: { validator: (v) => Number.isFinite(v) } },
  calories: { type: Number, required: true, min: 0,
    validate: { validator: (v) => Number.isFinite(v) } },
  expirationDate: { type: Date, required: false },
  minimumThreshold: { type: Number, required: false, min: 0,
    validate: { validator: (v) => Number.isFinite(v) } },
}, { timestamps: true });
```

A `User.ts` séma a jelszó biztonságos kezeléséhez `pre('save')` Mongoose hook-ot alkalmaz: beállításakor és módosításakor a jelszó automatikusan `bcrypt`-tel titkosítódik (10 sókör). A `comparePassword` instance metódus a bejelentkezéskor ellenőrzi a jelszót a tárolt hash-sel szemben anélkül, hogy a hash bármikor visszakerülne a klienshez (`select: false`).

models/User.ts — bcrypt jelszókezelés

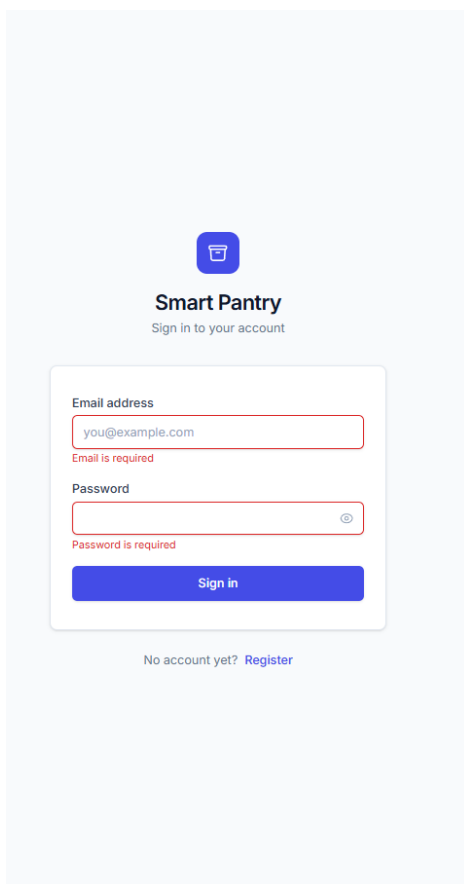
```
// models/User.ts — pre('save') hook és comparePassword metódus
userSchema.pre('save', async function (next) {
  if (!this.isModified('password')) return next();
  const salt = await bcrypt.genSalt(SALT_ROUNDS); // 10 sókör
  this.password = await bcrypt.hash(this.password, salt);
  next();
});

userSchema.methods.comparePassword = async function (
  candidatePassword: string
): Promise<boolean> {
  return bcrypt.compare(candidatePassword, this.password);
};
```

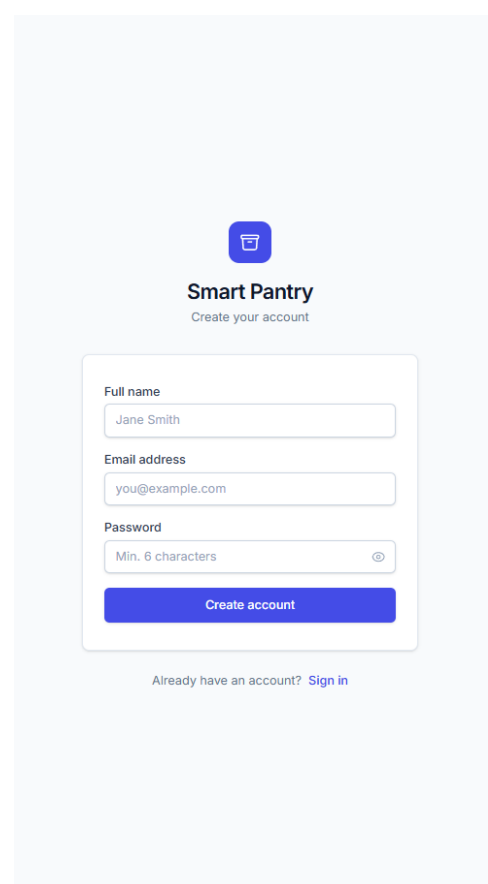
5. Képernyők

5.1 Bejelentkezés és regisztráció

A bejelentkezési és regisztrációs képernyő a nem hitelesített felhasználókat fogadja. Mindkét mód egyetlen, kártyastílusú komponensen belül váltható, reaktív Angular Forms validációval. Az e-mail cím regex-mintával, a jelszó minimum 6 karakterrel van ellenőrizve. Sikeres bejelentkezés vagy regisztráció után az authInterceptor automatikusan minden API-kéréshez csatolja a JWT token, és a felhasználó a dashboard-ra kerül.



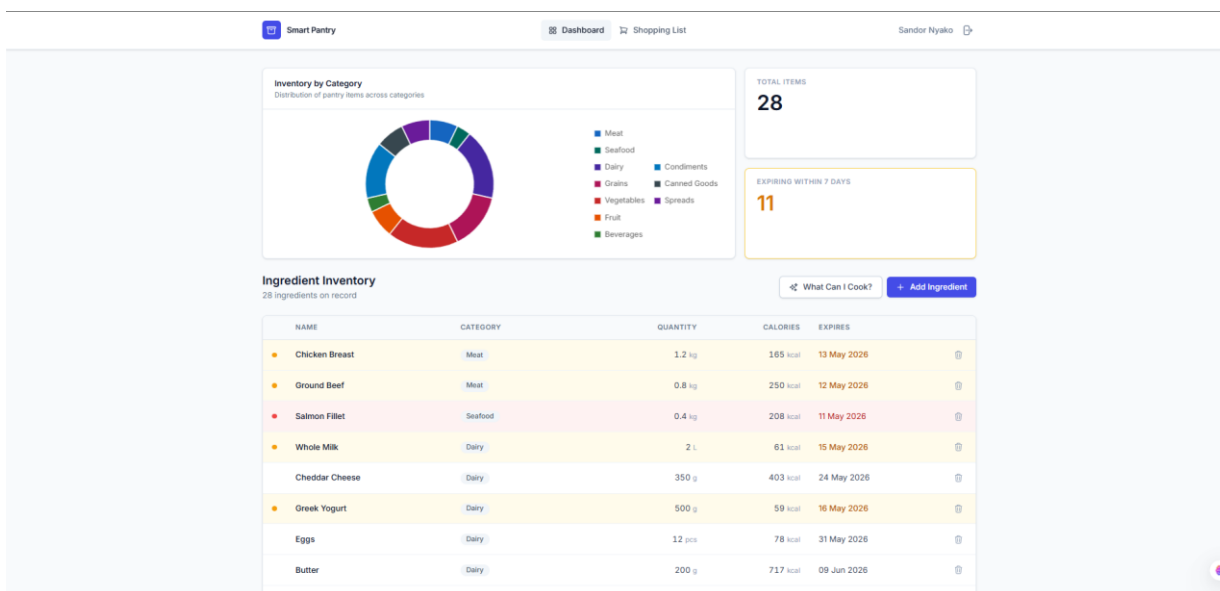
The image shows the 'Sign in' screen for 'Smart Pantry'. At the top, there is a blue icon of a shopping basket and the text 'Smart Pantry' followed by 'Sign in to your account'. Below this is a white card with a red border. Inside the card, there are two input fields: 'Email address' with the value 'you@example.com' and a red error message 'Email is required' below it; and 'Password' with a red error message 'Password is required' below it. At the bottom of the card is a blue button labeled 'Sign in'. Below the card, there is a link that says 'No account yet? Register'.



The image shows the 'Create account' screen for 'Smart Pantry'. At the top, there is a blue icon of a shopping basket and the text 'Smart Pantry' followed by 'Create your account'. Below this is a white card with a red border. Inside the card, there are three input fields: 'Full name' with the value 'Jane Smith'; 'Email address' with the value 'you@example.com'; and 'Password' with the value 'Min. 6 characters' and a red error message 'Min. 6 characters' below it. At the bottom of the card is a blue button labeled 'Create account'. Below the card, there is a link that says 'Already have an account? Sign in'.

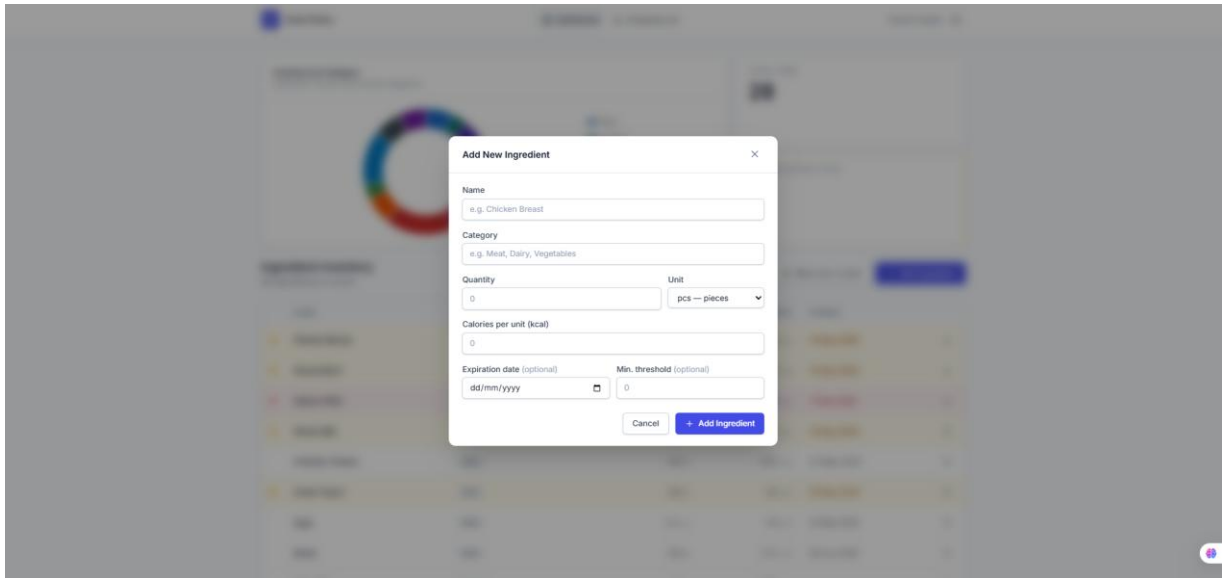
5.2 Főoldal (Dashboard)

A főoldalon felül statisztika-kártyák mutatják az összes alapanyag számát és a hamarosan lejáró tételek számát. Alatta a doughnut diagram a kategória-eloszlást jeleníti meg. A 'What Can I Cook?' gomb a Spoonacular receptjavaslatokat tölti be. Az alapanyag-táblázat sorait háromfajta bal oldali színes pont jelzi: szürke (normál), sárga (7 napon belül lejár), piros (lejárt). A táblázat fejléce és az 'Add Ingredient' gomb a jobb felső sarokban helyezkedik el.



5.3 Alapanyag hozzáadása — modális ablak

Az 'Add Ingredient' gombra kattintva egy centrált modális ablak nyílik meg, backdrop-blur effekttel és animált megjelenéssel. Az ablak az Escape billentyűre és a háttérre kattintásra bezárható. A form hét mezőt tartalmaz: Name, Category, Quantity, Unit (legördülő: pcs, g, kg, ml, l, cups, tbsp, tsp), Calories, Expiration Date és Minimum Threshold. Mentés közben a gomb 'Saving...' feliratot és forgó spinnert mutat, dupla-beküldés elleni védelemmel. Sikeres mentéskor toast-értesítés jelenik meg és az elem azonnal megjelenik a táblázat tetején.



5.4 Bevásárlólista (Shopping List)

A bevásárlólista képernyő a navigációs sávból érhető el. Automatikusan azokat az alapanyagokat listázza, amelyek mennyisége a beállított minimumThreshold alá esett. A táblázat oszlopai: Alapanyag neve, Jelenlegi mennyiség, Minimum szükséges, Hiány és 'Mark as Purchased' gomb. A gombra kattintva PUT-kérés megy a backend felé, amely a mennyiséget a minimumThreshold értékre állítja; sikeres válasz után az elem eltűnik a listáról és toast-értesítés jelenik meg.

Smart Pantry Dashboard Shopping List Sándor Nyeko

Shopping List
Items below their minimum stock threshold

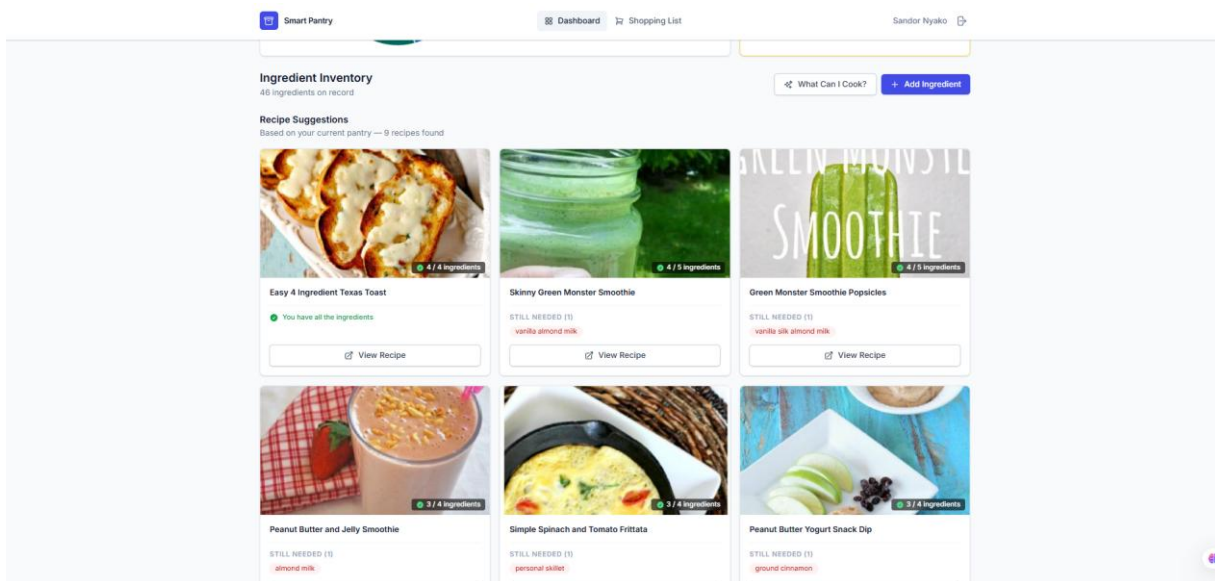
Refresh

ITEM	CATEGORY	IN STOCK	MIN. REQUIRED	DEFICIT	
Joghurt	Tejtermék	0 db	4 db	4 db	Mark as Purchased
Kenyér	Élelmiszer	0 db	2 db	2 db	Mark as Purchased
Paradicsom	Zöldség	1 db	5 db	4 db	Mark as Purchased
Rizs	Gabona	50 g	500 g	450 g	Mark as Purchased
Tej	Tejtermék	0.5 l	3 l	2.5 l	Mark as Purchased
Tojás	Tejtermék	2 db	10 db	8 db	Mark as Purchased
Vaj	Tejtermék	30 g	200 g	170 g	Mark as Purchased

"Csirkemell" restocked to 500 g.

5.5 Receptjavaslatok

A dashboard 'What Can I Cook?' gombjára kattintva az alkalmazás GET /api/recipes/suggest kérést küld a backendnek, amely a MongoDB-ből lekéri az összes alapanyagnevet, majd proxy-ként hívja a Spoonacular /recipes/findByIngredients végpontot. A frontend CSS Grid elrendezésben kártyákon jeleníti meg a recepteket: receptfotó, cím és hiányzó hozzávalók száma. A kártyákra kattintva a recept részletes oldala nyílik meg a Spoonacular weboldalán. Ha a lekérdezés megghiúsul (pl. lejárt API kulcs), toast-értesítés tájékoztat a hibáról.



5.6 Toast értesítések

A globális ToastService (providedIn: 'root') signal-alapú értesítési rendszert biztosít az egész alkalmazásban. A toastok a képernyő jobb alsó sarkában jelennek meg animált slide-in és slide-out effektekkel. Zöld háttér jelzi a sikeres műveleteket (mentés, törlés, feltöltés), piros háttér a hibákat (API hiba, validációs hiba). Minden toast 4500 ms után automatikusan eltűnik, de manuálisan is bezárható az 'x' gombbal. A kétlépéses dismissal (exiting: true + setTimeout) CSS animáció után távolítja el a toast-ot a DOM-ból.